Getting Started with Ansible

Connecting to an EC2 Instance Using Amazon EC2 Instance Connect

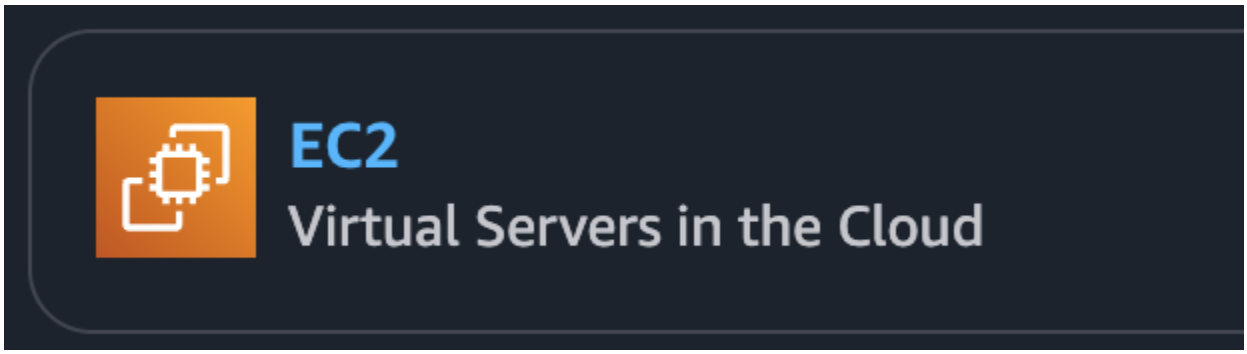
Introduction

In this lab step, you will connect to an Amazon EC2 instance named

cloudacademylabs
using your web browser and Amazon EC2 Instance Connect.

Instructions

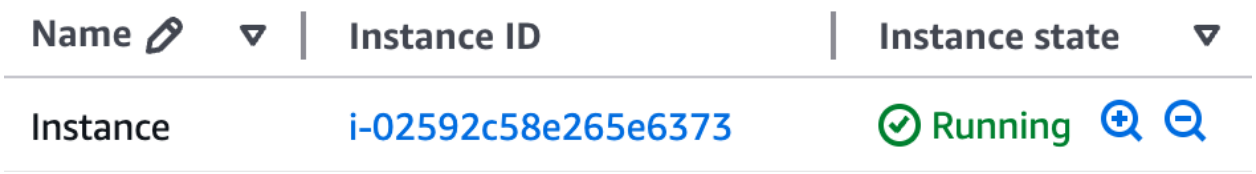
1. In the AWS Management Console search bar, enter *EC2*, and click the **EC2** result under **Services**:



2. To see available instances, click **Instances** in the left-hand menu:

The instances list page will open, and you will see an instance listed named

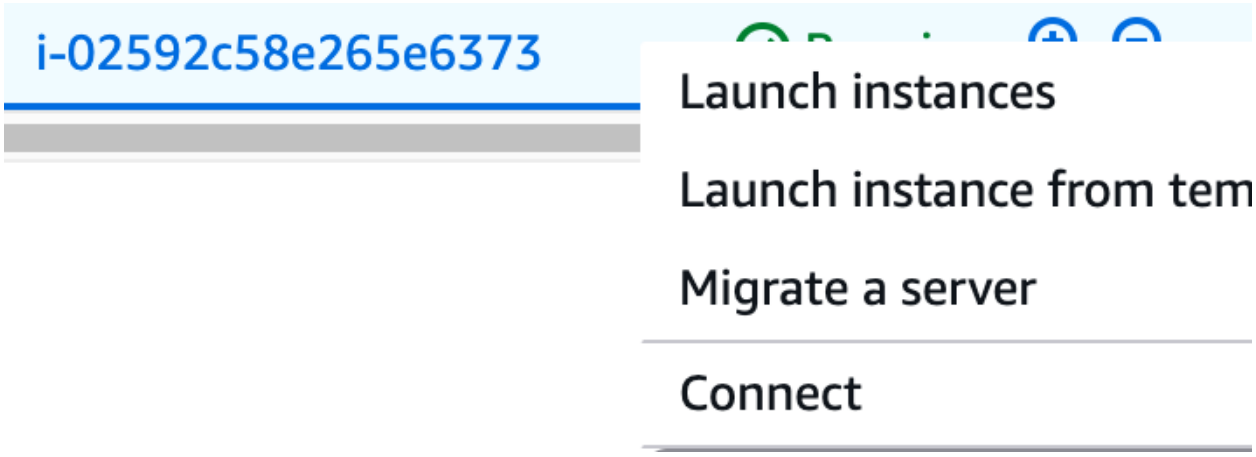
cloudacademylabs
listed:



If you don't see a running instance then the lab environment is still loading. Wait until the **Instance state** is **Running** before proceeding.

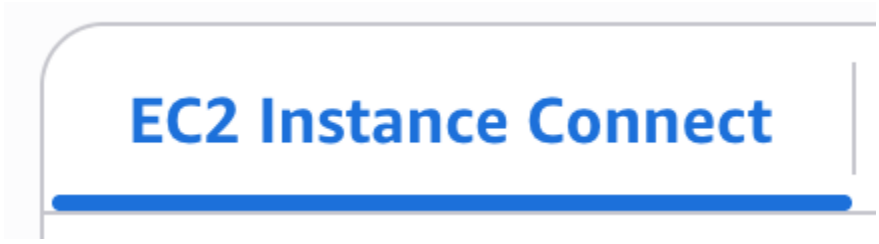
3. Right-click the instance named

cloudacademylabs
, and click **Connect**:



The **Connect to instance** form will load.

4. In the form, ensure the **EC2 Instance Connect** tab is selected:



You will see the instance's **Instance ID** and **Public IPv4 address** displayed.

5. In the **User name** textbox, enter

ubuntu
.

6. To connect to the instance using a browser-based shell, click **Connect**:

Connect

Creating a Basic Ansible Playbook

0/1

0 out of 1 validations checks passed

Introduction

In this lab step, you will create an Ansible playbook that starts a web server (Apache) and a database server (Mysql). This is the foundation of a LAMP (Linux Apache Mysql PHP) stack.

Instructions

1. To check that Ansible is installed, in the terminal, enter the following command:

Copy code

```
ansible --version
```

You will see output similar to:

```
ansible 2.9.6
  config file = /etc/ansible/ansible.cfg
  configured module search path = ['/home/ubuntu/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  executable location = /usr/bin/ansible
  python version = 3.8.10 (default, Sep 28 2021, 16:10:42) [GCC 9.3.0]
```

In this lab, you are using Ansible on a Ubuntu Linux server.

2. To start editing a new file, in the terminal, enter the following command:

Copy code

```
nano lamp.yml
```

Nano is a text editor that runs in the terminal.

The file is initially empty. You will populate it with a YAML playbook next.

Be aware that YAML syntax is sensitive to whitespace. In YAML, the level of indentation matters as does whether you use spaces in tabs. To avoid errors, use spaces instead of tabs to indent YAML lines.

3. Copy the following YAML and paste it into nano:

Copy code

```
---
- hosts: localhost
 gather_facts: false
 connection: local
 become: yes



 tasks:
   - name: Install our packages
     apt:
       name: ['apache2', 'mysql-server', 'mysql-common',
'mysql-client']
       state: present
       update_cache: true
```

Note: To preserve the indentation when pasting text into the nano text editor, press *Command+Control+V* (Mac OSX) or *Ctrl+Shift+V* (Windows).

This playbook consists of one task that installs four packages.

The task uses Ansible's built-in apt module to interact with the apt package manager which is used by the Ubuntu Linux operating system. To learn more about the apt Ansible module, see the ansible.builtin.apt – Manages apt-packages

, opens in a new tab

page of the Ansible documentation.

4. To exit nano, press *CTRL+X*.

You will be asked if you want to save the file.

5. To save the file, enter *Y*.

You will be asked to confirm the filename.

6. Press *enter* to confirm the **lamp.yml** filename.

You will be returned to the terminal command prompt.

```
ubuntu@ip-10-10-10-40:~$ ansible --version
ansible 2.9.6
  config file = /etc/ansible/ansible.cfg
  configured module search path = ['/home/ubuntu/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  executable location = /usr/bin/ansible
  python version = 3.8.10 (default, Sep 28 2021, 16:10:42) [GCC 9.3.0]
ubuntu@ip-10-10-10-40:~$ nano lamp.yml
ubuntu@ip-10-10-10-40:~$
```

i-08311fbb12101f741 (cloudacademylabs)

PublicIPs: 44.244.132.196 PrivateIPs: 10.10.10.40

Next, you will run your playbook.

7. Before you run your playbook, verify that the required packages are not currently installed by issuing the following command:

Copy code

```
dpkg -l apache2 mysql-server
```

You will see the following output confirming the packages aren't installed:

```
dpkg-query: no packages found matching apache2
dpkg-query: no packages found matching mysql-server
```

8. To run your Ansible playbook, enter the following command:

Copy code

```
ansible-playbook lamp.yml
```

Note: Because the inventory file isn't configured you'll see a warning. However, it doesn't matter when using Ansible to configure localhost.

You will see output that ends with the following:

```
PLAY RECAP *********************************************************************************************
localhost                  : ok=1    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

In this lab, you are using Ansible to configure the local server that you run the playbook on. Ansible can be used to manage and configure remote servers.

```
ubuntu@ip-10-10-10-40:~$ ansible-playbook lamp.yml
[WARNING]: provided hosts list is empty, only localhost is available. Note that the implicit localhost does not match 'all'

PLAY [localhost] ***************************************************************************************

TASK [Install our packages] ****************************************************************************

changed: [localhost]

PLAY RECAP *********************************************************************************************
localhost                  : ok=1    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

ubuntu@ip-10-10-10-40:~$
ubuntu@ip-10-10-10-40:~$ dpkg -l apache2 mysql-server
Desired=Unknown/Install/Remove/Purge/Hold
| Status=Not/Inst/Conf-files/Unpacked/halF-conf/Half-inst/trig-aWait/Trig-pend
|/ Err?=(none)/Reinst-required (Status,Err: uppercase=bad)
||/ Name           Version                 Architecture Description
+++-==============-=======================-============-===================================================================
ii  apache2        2.4.41-4ubuntu3.23      amd64        Apache HTTP Server
ii  mysql-server   8.0.42-0ubuntu0.20.04.1 all          MySQL database server (metapackage depending on the latest version)
ubuntu@ip-10-10-10-40:~$
```

i-08311fbb12101f741 (cloudacademylabs)

PublicIPs: 44.244.132.196 PrivateIPs: 10.10.10.40

CloudShell Feedback Console Mobile App © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

9. To confirm that the packages have been installed, re-run the `dpkg` command:

Copy code

```
dpkg -l apache2 mysql-server
```

This time you will see output ending with the following, confirming that the packages have been installed:

```
||/ Name           Version                 Architecture Description
+++-==============-=======================-============-===================================================================
ii  apache2        2.4.41-4ubuntu3.8       amd64        Apache HTTP Server
ii  mysql-server   8.0.27-0ubuntu0.20.04.1 all          MySQL database server (metapackage depending on the latest version)
```

Ensuring Services are Started in your Ansible Playbook

Introduction

In this lab step, you will modify your Ansible playbook to start the Apache webserver and Mysql server that you installed.

Instructions

1. Open your playbook file in nano:

Copy code

```
nano lamp.yml
```

2. At the end of the file, add the following:

Copy code

```
- name: Confirm services are running
    service:
      name: "{{ item }}"
      state: started
    with_items:
      - apache2
      - mysql
```

Your full playbook file will look like this:

```
---
- hosts: localhost
  gather_facts: false
  connection: local
  become: yes

  tasks:
    - name: Install our packages
      apt:
        name: ['apache2', 'mysql-server', 'mysql-common', 'mysql-client']
        state: present
        update_cache: true
    - name: Confirm services are running
      service:
        name: "{{ item }}"
        state: started
      with_items:
        - apache2
        - mysql
```

The task you have added uses Ansible's built-in service module. To learn more about this module, see the ansible.builtin.service – Manage services

, opens in a new tab

page of the Ansible documentation.

3. To exit and save your playbook file, Press *CTRL-X*, press *Y*, and press *enter*.

4. To start the Apache and Mysql services, re-run your playbook:

Copy code

```
ansible-playbook lamp.yml
```

You will see the following output:

```
ubuntu@ip-10-10-10-40:~$ ansible-playbook lamp.yml
[WARNING]: provided hosts list is empty, only localhost is available. Note that the implicit localhost does not match 'all'

PLAY [localhost] ***************************************************************************************************************************

TASK [Install our packages] ****************************************************************************************************************
ok: [localhost]

TASK [Confirm services are running] ********************************************************************************************************
ok: [localhost] => (item=apache2)
ok: [localhost] => (item=mysql)

PLAY RECAP *********************************************************************************************************************************
localhost                  : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

ubuntu@ip-10-10-10-40:~$ curl http://checkip.amazonaws.com
44.244.132.196
ubuntu@ip-10-10-10-40:~$
```

5. To retrieve the IP address of the server, enter the following command:

Copy code

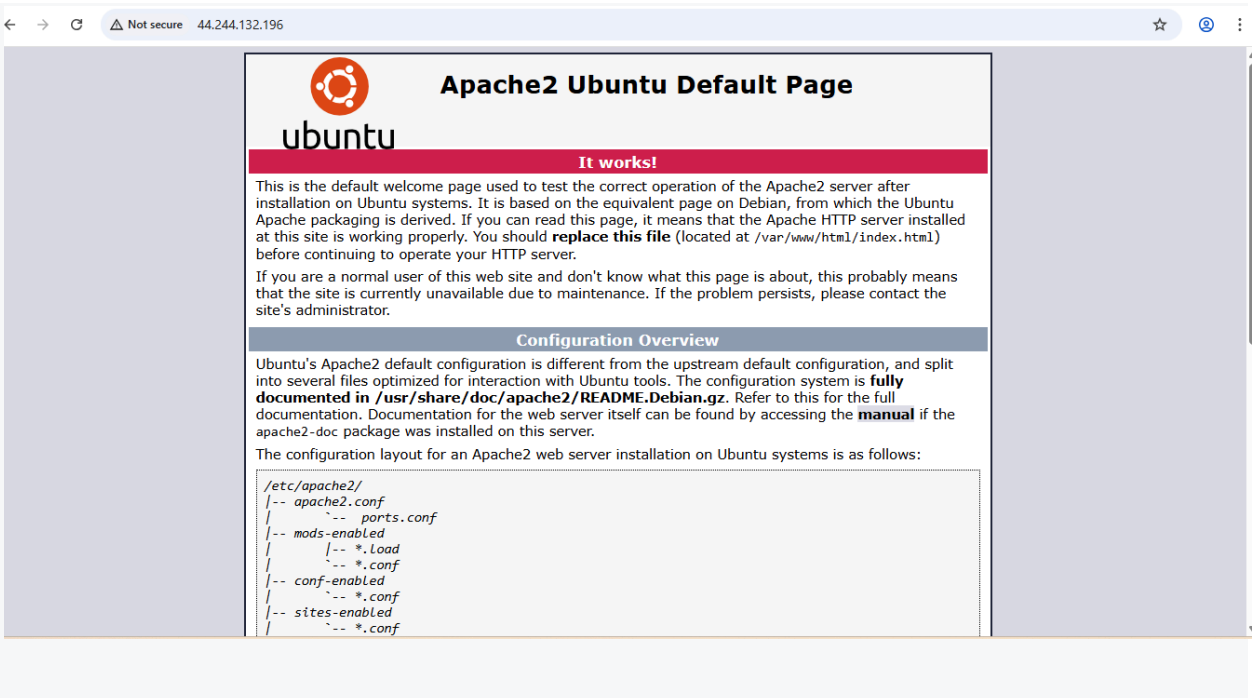
```
curl http://checkip.amazonaws.com
```

You will see the public IPv4 address of the server printed.

This is an unofficial service provided by AWS to determine the public IP address of a host. You can also find the IPv4 address in the Amazon EC2 console.

6. Visit the IP address in a new browser tab.

You will see the test page of the Apache webserver displayed:



This page confirms that the webserver is running.

Adding Variables to your Ansible Playbook

0/1

0 out of 1 validations checks passed

Introduction

Ansible allows you to use variables to make your playbooks more reusable because you can edit the variables at runtime.

In this lab step, you will add variables to your playbook to make your playbook easy to reuse.

Instructions

1. To open your file, in the terminal, enter the following:

Copy code

```
nano lamp.yml
```

The services and packages are currently both static, you will create variables for these.

2. To remove everything in the file, with your cursor on the first line, hold *CTRL+K* until all the text is gone.

CTRL+K is nano's cut command.

3. Paste the following into your empty file:

Copy code

```
---
- hosts: localhost
 gather_facts: false
 connection: local
 become: yes


 vars:
   packages:
     - apache2
     - mysql-server
```

```
    - mysql-common
    - mysql-client
  services:
    - apache2
    - mysql


tasks:
  - name: Install our packages
    apt:
      name: "{{ packages }}"
      state: present
  - name: Confirm services are running
    service:
      name: "{{ item }}"
      state: started
    with_items: "{{ services }}"
```

You've defined a `vars` YAML mapping with sub mappings for `packages` and `services`. The value of the sub mappings are arrays. The arrays contain the values that you had hard-coded in the task definitions previously.

4. To exit and save your playbook file, Press *CTRL-X*, press *Y*, and press *enter*.

5. To re-run your playbook, in the terminal, enter the following:

Copy code

```
ansible-playbook lamp.yml
```

You will see the same output as before. The playbook is functionally the same, except now you are using variables to define the services and packages.

By default, Apache only has HTTP enabled. To enable an HTTPS site in Apache, you need to run the following commands:

Copy code

```
sudo a2enmod ssl
sudo a2ensite default-ssl
```

```
TASK [Confirm services are running] ***********************************************************************************************************
ok: [localhost] => (item=apache2)
ok: [localhost] => (item=mysql)

PLAY RECAP ************************************************************************************************************************************
localhost                  : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

ubuntu@ip-10-10-10-40:~$ sudo a2enmod ssl
Considering dependency setenvif for ssl:
Module setenvif already enabled
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Enabling module socache_shmcb.
Enabling module ssl.
See /usr/share/doc/apache2/README.Debian.gz on how to configure SSL and create self-signed certificates.
To activate the new configuration, you need to run:
  systemctl restart apache2
ubuntu@ip-10-10-10-40:~$ sudo a2ensite default-ssl
Enabling site default-ssl.
To activate the new configuration, you need to run:
  systemctl reload apache2
```

***Note*: Don't run these commands manually, you will make ansible do that for you next.**

These commands enable the Apache SSL module and the default HTTPS site for the Apache webserver.

6. Open your lamp.yml file with `nano lamp.yml`.

7. At the bottom of the file, add the following:

Copy code

```
- name: EnableApache2 modssl
    shell: a2enmod ssl
  - name: EnableApache2 DefaultHTTPS site
    shell: a2ensite default-ssl
```

The tasks section of your file will look like this:

```
tasks:
  - name: Install our packages
    apt:
      name: "{{ packages }}"
      state: present
  - name: Confirm services are running
    service:
      name: "{{ item }}"
      state: started
    with_items: "{{ services }}"
  - name: Enable Apache2 modssl
    shell: a2enmod ssl
  - name: Enable Apache2 Default HTTPS site
    shell: a2ensite default-ssl
```

These tasks use Ansible's shell service to run shell commands on the server.

8. To exit and save your playbook file, Pres *CTRL-X*, press *Y*, and press *enter*.

9. To enable the default HTTPS site, re-run `ansible-playbook lamp.yml`.

You will see output that ends with the following:

```
ubuntu@ip-10-10-10-40:~$ ansible-playbook lamp.yml
[WARNING]: provided hosts list is empty, only localhost is available. Note that the implicit localhost does not match 'all'

PLAY [localhost] ***********************************************************************************************************************************

TASK [Install our packages] ************************************************************************************************************************
ok: [localhost]

TASK [Confirm services are running] ****************************************************************************************************************
ok: [localhost] => (item=apache2)
ok: [localhost] => (item=mysql)

TASK [EnableApache2 modssl] ************************************************************************************************************************
changed: [localhost]

TASK [EnableApache2 DefaultHTTPS site] *************************************************************************************************************
changed: [localhost]

PLAY RECAP *****************************************************************************************************************************************
localhost                  : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

ubuntu@ip-10-10-10-40:~$
```

If you return to your site with the Apache webserver's default site open and prefix the IP address with `https://` you will see a message that the site can't be reached. This

is because after enabling the SSL module and the HTTPS site, Apache needs to be restarted for the changes to take effect.

10. Open your lamp.yml file in nano again and add the following at the end of the playbook:

Copy code

```
- name: Restart Apache
    service:
      name: apache2
      state: restarted
```

This uses the same `service` Ansible module that you use elsewhere in the playbook to confirm that the Apache and Mysql services are running. In this case though, you set the `state` to `restarted`. This tells Ansible to restart the Apache webserver instead of just ensuring that it was started.

11. To exit and save your playbook file, Pres *CTRL-X*, press *Y*, and press *enter*.

12. Re-run your Ansible playbook with `ansible-playbook lamp.yml`.

You will see output that ends with:

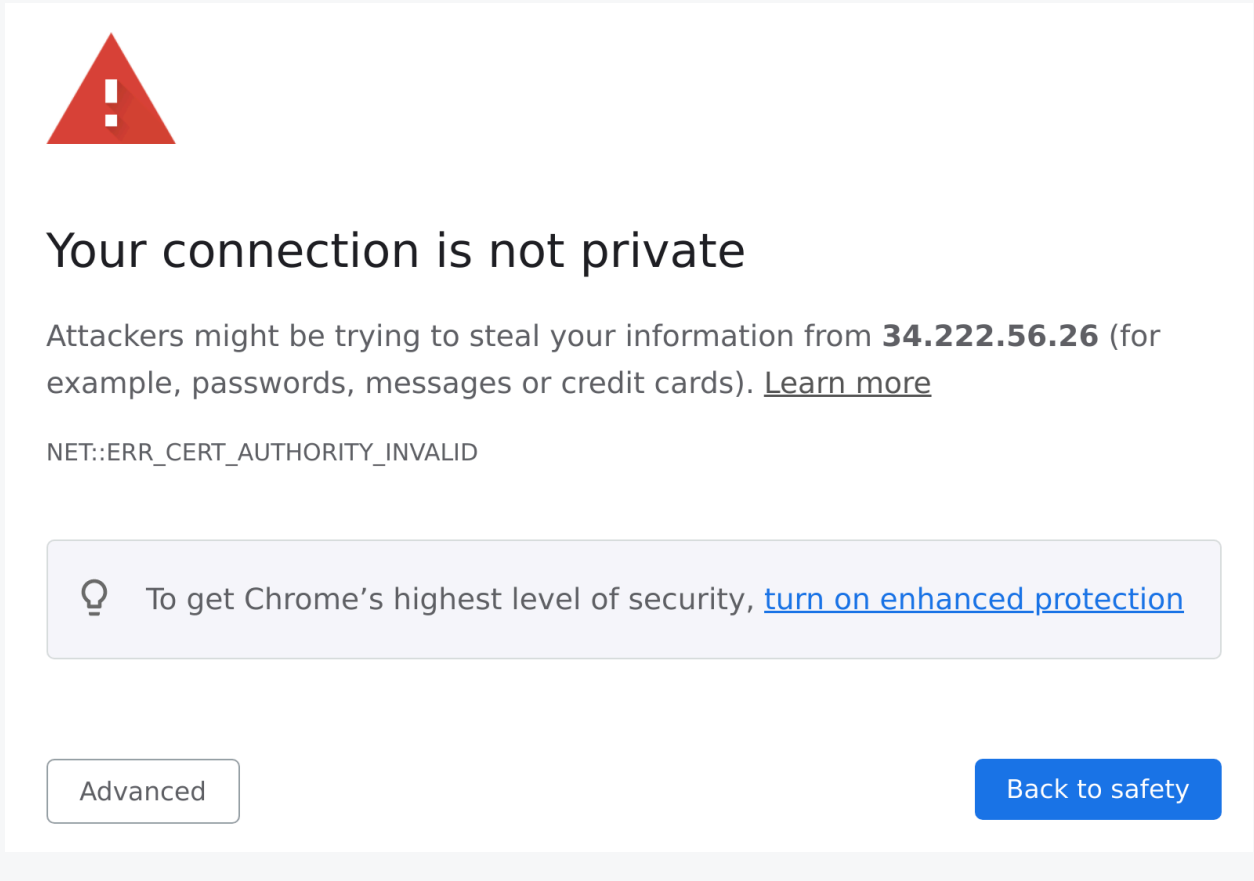
```
TASK [Restart Apache] ************************
changed: [localhost]

PLAY RECAP ***********************************
localhost                  : ok=5    changed=3
```

Notice that Ansible returned changed for the tasks to enable the SSL module and the HTTPS site. This is because even though they are already enabled, Ansible does not know this and cannot detect it.

13. Return to your browser tab with the IP address of the server open, add *https://* at the beginning in the address bar, and press *enter*.

This time you will see a message in your browser that the connection is insecure. This is because although you have enabled an HTTPS site, the server doesn't have a valid SSL certificate.

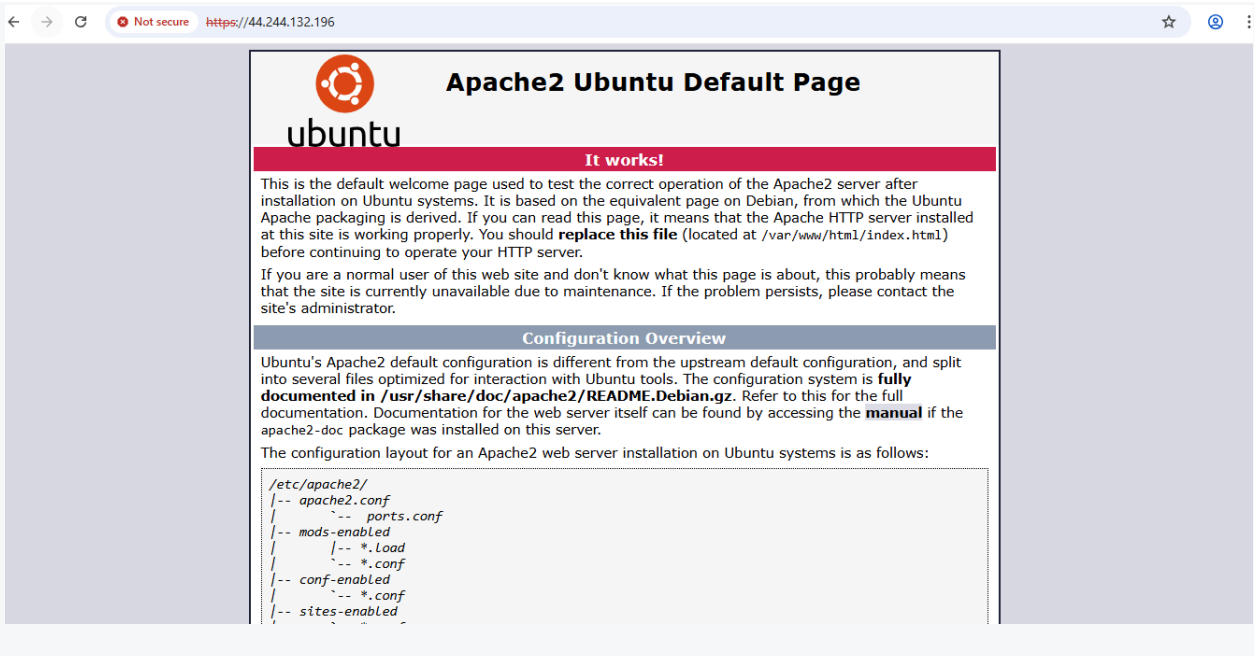
The exact message you see will depend upon your web browser. Here is how Chrome handles it:



14. If using Chrome, click Advanced, and then Proceed to xxx.xxx.xxx.xxx (unsafe).

Other browsers may have a different UI but should still allow you to visit an insecure site at your own risk. It's safe in this lab because you are navigating to a web server that you created and you won't input or send any sensitive data.

You will see the default Apache page again:



You have successfully enabled the default HTTPS Apache site.

Summary

In this lab, you created an Ansible YAML playbook using the nano terminal-based text editor. Your playbook installed the Apache webserver and Mysql database packages. You then modified your playbook to use variables enabling easier re-use. Finally, you modified your playbook to configure the Apache webserver to enable a HTTPS site.

See recommendations | Restart lab | Bookmark

Did you enjoy this hands-on lab session?

Completion	Time Spent	Checks
100%	28m 32s	Failed 0 Passed 2 Attempts 2

Validation overview | Updated skills | Recommendations

Step 3: Creating a Basic Ansible Playbook

Installed the Apache Web Server Package — Passed

Step 5: Adding Variables to your Ansible Playbook

Enabled the SSL Module for the Apache Web Server — Passed

60°F Partly cloudy | Search | ENG | 10:10 PM 11/13/2025